

HADOOP MAPREDUCE EXECUTION FLOW

How to run a Hadoop MapReduce word count program, step by step.

1. What is happening here?

You are running a **MapReduce program** to count how many times each word appears in a file.

Example input file:

hello world

hello hadoop

world hadoop

Output:

hello 2

world 2

hadoop 2

This is the classic **Word Count program** in Hadoop.

2. Internal Flow of Execution

The notes show this flow:

Map

Combiner

Shuffle

Sort

Reducer

Let's understand each.

Step 1: Map

Mapper reads input line by line.

Input:

hello world

Mapper converts into:

(hello,1)

(world,1)

Another line:

hello hadoop

Becomes:

(hello,1)

(hadoop,1)

So mapper's job:

Break text into words and assign 1

Step 2: Combiner (Optional mini reducer)

Combiner does local aggregation.

Instead of sending:

(hello,1)

(hello,1)

(hello,1)

It combines locally into:

(hello,3)

Why?

To reduce network traffic.

This makes Hadoop faster.

Step 3: Shuffle

Hadoop groups same keys together.

Example:

Before shuffle:

(hello,1)

(world,1)

(hello,1)

(world,1)

After shuffle:

hello → [1,1]

world → [1,1]

Step 4: Sort

Hadoop sorts keys.

Example:

apple

cat

dog

hello

Sorting helps reducer process systematically.

Step 5: Reducer

Reducer adds values.

Example:

hello → [1,1,1]

Reducer:

hello 3

This is final output.

3. "MapReduce code is not necessary here"

Your note says:

mapreduce code is not necessary here

Meaning:

For execution, you don't write code again.

You already have compiled file:

mapreduce_prog.jar

You only run it.

4. What is JAR?

JAR = Java Archive

It contains compiled Java classes.

Think of it like:

- Source code → .java
- Compiled → .class
- Packaged → .jar

So:

mapreduce_prog.jar

contains mapper + reducer logic.

5. Creating Input File on Gateway Node

The notes show:

pwd

`mkdir data`

`cd data`

`mkdir input`

`cd input`

`vi inputfile.txt`

What this does

`pwd`

Shows current directory.

`mkdir data`

Creates folder:

`data`

`cd data`

Moves into data folder.

`mkdir input`

Creates input folder.

`cd input`

Moves inside input folder.

`vi inputfile.txt`

Creates and opens file.

You type text inside:

`hello hadoop`

hello spark

Save file.

To check content:

cat inputfile.txt

Displays file content.

6. Why Move File to HDFS?

Important concept.

The file is first created on:

Gateway node (local Linux machine)

But Hadoop processes files from:

HDFS

So we must upload it.

7. Creating Folder in HDFS

Notes show:

hadoop fs -mkdir data

hadoop fs -mkdir data/input

This creates folders inside HDFS.

HDFS structure:

/user/itv005857/data/input

8. Uploading File to HDFS

Command:

hadoop fs -put /home/itv005857/data/input/inputfile.txt
/user/itv005857/data/input

Meaning:

Source

Local file:

/home/itv005857/data/input/inputfile.txt

Destination

HDFS:

/user/itv005857/data/input

Simple meaning:

Copy local file into Hadoop storage

9. Running the MapReduce Job

General syntax:

hadoop jar <jarname> <input_path> <output_directory>

Your example:

hadoop jar mapreduce_prog.jar /user/itv005857/data/input/inputfile.txt
/user/itv005857/data/output

Breakdown:

hadoop jar

Run Hadoop program

mapreduce_prog.jar

Program file

inputfile.txt

Input file

output

Where result will be stored

Important Rule

Output directory must NOT already exist.

Why?

Because Hadoop prevents overwriting.

If it exists:

Job fails.

10. What Happens After Running?

Hadoop creates output folder automatically.

Check:

```
hadoop fs -ls data/output
```

You'll see:

```
_SUCCESS
```

```
part-r-00000
```

11. What is _SUCCESS?

It is an empty file.

It tells:

Job completed successfully

Think of it like a success flag.

12. What is part-r-00000?

This contains actual output.

Example:

```
hello 2
```

```
hadoop 1
```

```
spark 1
```

Why r?

It means:

Reducer output

If multiple reducers:

part-r-00000

part-r-00001

part-r-00002

13. Reading Output

Command:

```
hadoop fs -cat data/output/part*
```

Why *?

Wildcard.

Means:

Open all files starting with part

Useful when multiple reducers exist.

14. Delete Output Folder

Since Hadoop won't overwrite output, remove old output before rerunning.

Command:

```
hadoop fs -rm -R /user/itv005857/data/output
```

Meaning:

- rm → remove
 - -R → recursive (delete folder + contents)
-

15. Complete Flow (Easy to Remember)

Step 1

Create local file

```
vi inputfile.txt
```

Step 2

Create HDFS folder

```
hadoop fs -mkdir data
```

```
hadoop fs -mkdir data/input
```

Step 3

Upload file

```
hadoop fs -put inputfile.txt data/input
```

Step 4

Run job

```
hadoop jar mapreduce_prog.jar input output
```

Step 5

Check output

```
hadoop fs -ls output
```

Step 6

View result

```
hadoop fs -cat output/part*
```

Step 7

Delete output if rerunning

```
hadoop fs -rm -R output
```

Most Important Interview Questions From These Notes

Why move file from gateway node to HDFS?

Because Hadoop processes data stored in HDFS.

Why output directory should not exist?

Hadoop avoids accidental overwrite.

What does `_SUCCESS` mean?

Job completed successfully.

What does `part-r-00000` mean?

Reducer output file.

Why use combiner?

To reduce data transfer between mapper and reducer.

When is reducer count 0 useful?

Exactly what you asked earlier.

Use **0 reducers** when you only need mapper output.

Example:

Filtering logs

Input:

INFO

ERROR

INFO

ERROR

Need only ERROR lines.

Mapper outputs:

ERROR

ERROR

No aggregation needed.

So reducer is unnecessary.

ZERO REDUCER CASE :

Yes — **running a Hadoop job with 0 reducers is absolutely valid**, and in some cases it's exactly what you *should* do. But you need to understand *why* you're doing it, otherwise you'll misuse it.

◆ What happens when reducers = 0?

- Hadoop runs **only the Mapper phase**
- Output is written **directly from Mappers to HDFS**
- **No shuffle & sort phase**
- Faster execution (no network transfer)

👉 Think of it like:

“Read → Filter/Transform → Write” (no grouping, no combining)

◆ When 0 reducers is the RIGHT choice

Use **0 reducers** when your logic is **independent per record** (no aggregation needed).

✓ 1. Filtering data

Example:

WHERE age > 25

- ✓ Each record is checked independently
 - ✓ No need to combine data
-

✓ 2. Data cleaning / transformation

- Removing nulls
 - Changing formats
 - Extracting fields
-

✓ 3. Log processing

- Extract errors from logs
 - Parse JSON/XML records
-

✓ 4. ETL preprocessing

- Preparing raw data before aggregation jobs
-

✓ 5. Simple projections

SELECT name, salary FROM employees;

● When 0 reducers is a BAD idea

If your problem needs **data to come together**, mapper-only will break logic.

✗ 1. Aggregations

COUNT(*), SUM(salary), AVG(age)

👉 Requires combining data → needs reducer

✗ 2. GROUP BY

GROUP BY department

👉 Data must be shuffled and grouped

❌ 3. Joins

- Inner join
 - Outer join
 - 👉 Data from different sources must meet → reducer needed
-

❌ 4. Sorting globally

👉 Mappers only sort locally, not across dataset

🏛️ Quick Decision Table

Use Case	Reducers Needed?
Filtering	❌ No
Data cleaning	❌ No
Parsing logs	❌ No
Aggregation	✅ Yes
GROUP BY	✅ Yes
Joins	✅ Yes
Global sorting	✅ Yes

🔥 Real Insight (Important)

Running with 0 reducers:

- 🚀 Faster (no shuffle phase)
- 💰 Less resource usage
- ⚠️ But produces **multiple output files** (one per mapper)

👉 If you need a **single final output file**, reducers help consolidate.

🧠 Simple Rule to Remember

If each record can be processed **independently** → **0 reducers**

If records need to be **combined** → **reducers required**

💡 Practical Example

Mapper-only (0 reducers)

```
if (age > 25) emit(record);
```

Mapper + Reducer

```
emit(department, salary);
```

```
// Reducer
```

```
sum all salaries per department
```

⚠️ Coaching Advice

Don't just think:

"Can I run with 0 reducers?"

Instead ask:

"Does my logic require data from multiple records to meet?"

That one question will save you in interviews **and** real projects.

Let's walk through a real Hadoop job like it's actually running on a cluster.

I'll simulate both cases:

- ✅ **Job A: Mapper-only (0 reducers)** → filtering logs
 - ✅ **Job B: Mapper + Reducer** → counting per key
-

🔗 CLUSTER SETUP (same for both)

Assume:

- 1 input file = **1 GB**
- HDFS block size = **128 MB**
- 👉 Total blocks = **8**
- 👉 So **8 Mappers** will run (1 per block)

Cluster:

- 1 NameNode
 - Multiple DataNodes
-

JOB A: 0 REDUCERS (Mapper-only Job)

Use case: Filter ERROR logs

◆ **Step 1: Input stored in HDFS**

/logs/app.log (1 GB)

→ split into 8 blocks

→ distributed across DataNodes

◆ **Step 2: Job submission**

You set:

`setNumReduceTasks(0)`

👉 Hadoop understands:

“No reducer phase needed”

◆ **Step 3: Mapper execution (parallel)**

Each mapper processes **its own block locally**:

Mapper 1 → Block 1

Mapper 2 → Block 2

...

Mapper 8 → Block 8

Inside each Mapper:

```
if (log.contains("ERROR")) {  
    emit(log);  
}
```

👉 Important:

- No key grouping

- No dependency on other mappers
-

◆ Step 4: NO shuffle phase ❌

This is the biggest difference:

- ❌ No sorting
- ❌ No data transfer across network
- ❌ No grouping

👉 This is why it's FAST

◆ Step 5: Output written directly

Each mapper writes its own output:

/output/

part-m-00000

part-m-00001

...

part-m-00007

👉 8 files (one per mapper)

🔥 Flow Summary (0 reducers)

HDFS → Split → Mappers → HDFS Output

✓ Fast

✓ Cheap

⚠️ Multiple output files

❌ No aggregation possible

⚙️ JOB B: WITH REDUCERS

🎯 Use case: Count ERROR logs per service

◆ Step 1: Same input splitting

Still:

- 8 blocks → 8 mappers

◆ Step 2: Mapper execution

Each mapper emits key-value pairs:

```
emit(service_name, 1);
```

Example output:

Mapper 1:

Auth → 1

Payment → 1

Auth → 1

◆ Step 3: SHUFFLE & SORT (🔥 critical phase)

Now Hadoop does heavy lifting:

📦 What happens:

1. All "Auth" keys from ALL mappers go to same reducer
2. All "Payment" keys go to another reducer

👉 Data is:

- Transferred over network
- Sorted by key
- Grouped

Example:

Reducer 1 gets:

Auth → [1,1,1,1,1]

Reducer 2 gets:

Payment → [1,1,1]

◆ Step 4: Reducer execution

sum(values)

Output:

Auth → 5

Payment → 3

◆ Step 5: Final output

/output/

part-r-00000

part-r-00001

👉 Fewer, aggregated files

🔥 Flow Summary (with reducers)

HDFS → Split → Mappers → Shuffle/Sort → Reducers → Output

✂️ SIDE-BY-SIDE DIFFERENCE

Stage	0 Reducers	With Reducers
Mapper	✅ Yes	✅ Yes
Shuffle	❌ No	✅ Yes
Network usage	❌ Minimal	🔥 High
Sorting	❌ No	✅ Yes
Aggregation	❌ No	✅ Yes
Output files	Many	Fewer
Speed	🚀 Fast	🐢 Slower

🧠 WHAT PEOPLE MISS (Important Insight)

👉 Even if your logic *looks simple*, ask:

“Do I need data from different blocks to come together?”

- If **NO** → use 0 reducers
 - If **YES** → reducers are mandatory
-

⚠️ Real-world mistake

People sometimes do:

- Filtering + aggregation in mapper only ❌

👉 Result:

- Wrong answers (because each mapper sees only partial data)
-

💡 Final Mental Model

Mapper-only job:

Independent pipelines (no communication)

Reducer job:

All roads lead to grouping point (communication happens)

LET'S MAKE IT CRYSTAL CLEAR WITH A REAL EXAMPLE USING 2 REDUCERS.

🧠 Key Idea First (Don't skip)

When you use **2 reducers**, Hadoop:

- 👉 Splits keys (words) into **2 groups**
 - 👉 Each group goes to a **different reducer**
 - 👉 Each reducer produces its **own output file**
-

📁 Step 1: Input File

Let's take a simple file:

big data is interesting

big data is trending technology

⚙️ Step 2: Mapper Output

Mapper converts words into:

(big,1)

(data,1)

(is,1)

(interesting,1)

(big,1)

(data,1)

(is,1)

(trending,1)

(technology,1)

Step 3: Shuffle + Partitioning (MOST IMPORTANT)

Now Hadoop decides:

👉 Which word goes to **Reducer 1**

👉 Which word goes to **Reducer 2**

This is done using a **hash function**

Example Split (for understanding)

Let's assume:

Reducer 1 gets:

big → [1,1]

data → [1,1]

Reducer 2 gets:

is → [1,1]

interesting → [1]

trending → [1]

technology → [1]

⚠️ This split is NOT random — it's based on hashing
But for learning, assume this distribution.

Step 4: Reducer Processing

Reducer 1 Output:

big 2

data 2

Reducer 2 Output:

is 2

interesting 1

trending 1

technology 1

Step 5: Final Output Files in HDFS

When you run with **2 reducers**, output folder will contain:

_SUCCESS

part-r-00000 ← Reducer 1 output

part-r-00001 ← Reducer 2 output

File Contents

part-r-00000

big 2

data 2

part-r-00001

is 2

interesting 1

trending 1

technology 1

Important Observations

1. Output is SPLIT

Unlike 1 reducer:

 You get **multiple output files**

2. Order is NOT guaranteed

You may NOT see:

big data is interesting...

Instead, it's split across files.

3. Each key goes to **ONLY ONE** reducer

👉 “big” will NOT appear in both reducers

👉 Hadoop ensures grouping is correct

4. Why use multiple reducers?

✅ **Faster processing**

Work is divided

✅ **Parallel execution**

Reducers run at same time

🔥 **Real Hadoop Command**

```
hadoop jar mapreduce_prog.jar input output -D mapreduce.job.reduces=2
```

🧠 **Mental Model (Very Important)**

Think like this:

Mapper → creates words

↓

Partitioner → decides reducer

↓

Reducer 1 → processes half words

Reducer 2 → processes remaining words

↓

Output → multiple files

🌟 **One Common Mistake**

People think:

“Each reducer processes all data”

❌ **WRONG**

👉 Each reducer processes only its assigned keys

Quick Interview Answer

Q: What happens when we use 2 reducers?

- 👉 Output is divided into 2 files
 - 👉 Keys are distributed using hash function
 - 👉 Each reducer processes a subset of keys
 - 👉 Parallel processing improves performance
-

Why and when we need custom _partitioner

Problem First: Data Skew (Why default partitioner fails)

Using default hash partitioner:

$\text{partition} = \text{hash}(\text{key}) \% \text{numReducers}$

Real issue:

If your data looks like this:

error error error error error

info info

debug

👉 "error" dominates

What happens?

Reducer Data

R0 info, debug

R1 error, error, error, error, error

- 👉 One reducer is overloaded
- 👉 Others are idle
- 👉 Job becomes **slow (bottleneck)**

Goal of Custom Partitioner

👉 Distribute **load evenly**, not just keys randomly

Strategy 1 (Most Common Interview Answer)

“Split heavy keys manually”

Instead of sending all "error" to one reducer:

👉 We break it like:

error_0

error_1

error_2

Custom Partitioner Logic

```
public class CustomPartitioner extends Partitioner<Text, IntWritable> {
```

```
    @Override
```

```
    public int getPartition(Text key, IntWritable value, int numReducers) {
```

```
        String word = key.toString();
```

```
        if (word.startsWith("error")) {
```

```
            return 0; // heavy key goes to reducer 0
```

```
        } else if (word.startsWith("info")) {
```

```
            return 1;
```

```
        } else {
```

```
            return 2 % numReducers;
```

```
        }
```

```
    }
```

```
}
```

❌ But this alone is NOT enough

If "error" is huge:

👉 Still goes to only one reducer → still skew

🔥 Real Fix (Important)

✅ Split heavy key into multiple partitions

Mapper modification:

Instead of:

(error,1)

We do:

(error_0,1)

(error_1,1)

(error_2,1)

⚙️ New Partitioner

```
public int getPartition(Text key, IntWritable value, int numReducers) {  
    String word = key.toString();  
  
    if (word.startsWith("error_0")) return 0;  
    if (word.startsWith("error_1")) return 1;  
    if (word.startsWith("error_2")) return 2;  
  
    return Math.abs(word.hashCode()) % numReducers;  
}
```

🎨 What Happens Now?

Before (BAD):

Reducer 1:

error → [1000000 values]

After (GOOD):

Reducer 0: error_0 → 300000

Reducer 1: error_1 → 350000

Reducer 2: error_2 → 350000

👉 Load distributed

👉 Faster execution

⚠️ Final Step (VERY IMPORTANT)

Now output looks like:

error_0 300000

error_1 350000

error_2 350000

We must merge:

👉 Final result:

error 1000000

This is done using:

👉 Second MapReduce job OR post-processing

🧠 Clean Interview Answer

If interviewer asks:

❓ **“How do you handle data skew in Hadoop?”**

Answer:

Default partitioning may send large keys to a single reducer causing skew.

To fix this, we identify heavy keys and split them into multiple subkeys in the mapper.

Then a custom partitioner distributes these subkeys across reducers.

Finally, results are merged to get the correct output.

🎯 When to Use Custom Partitioner

Use when:

- Few keys have **huge data**
 - Word count with skewed words (like "the")
 - Log analysis (error dominates)
 - Joins with uneven keys
-

⚡ Reality Check (Important Advice)

Don't blindly use custom partitioner.

👉 First try:

- Combiner
- Increasing reducers

👉 Only use custom partitioner when:

- Skew is clearly visible
-

🧠 Final Mental Model

Heavy key → split → distribute → process → merge

Using Combiner

🚀 What Combiner Does (in one line)

👉 Combiner reduces the amount of data sent to reducers → less network → faster job

📦 Without Combiner (Slow)

Input:

error error error error error

Mapper output:

(error,1)

(error,1)

(error,1)

(error,1)

(error,1)

👉 All 5 records are sent over network to reducer

👉 More data transfer = **slow**

⚡ With Combiner (Fast)

Combiner runs **inside mapper**.

It converts:

$(\text{error}, 1) \times 5$

👉 Into:

$(\text{error}, 5)$

🔥 What Changed?

Without Combiner With Combiner

5 records sent 1 record sent

More network Less network

Slower Faster

🧠 Why This Improves Speed

Because in Hadoop:

👉 **Network transfer is the biggest bottleneck**

Combiner reduces:

- Data size
- Network usage
- Shuffle time

👉 So overall job becomes **faster**

📊 Real Thinking

Imagine:

- 1 million $(\text{error}, 1)$ pairs
- Without combiner → send 1 million records
- With combiner → send maybe **1 record per mapper**

👉 Massive speed improvement

⚠️ Important

👉 Combiner is useful when:

- Same key repeats many times

👉 Not useful when:

- All keys are different

🎯 **Final Simple Line (Remember this)**

Combiner = **mini reducer in mapper** → reduces data → speeds up processing